

Licence 2^{ème} Année
Sciences pour l'ingénieur (SPI)

UE 3 – Informatique

Programmation

Chapitre 2 :
Les fonctions

Enseignant : Yves André CHAPUIS

1. Généralités

1.1. Pourquoi programmer avec des fonctions ?

En programmation, les **fonctions** sont très utiles pour :

- **réaliser** plusieurs fois la même opération au sein d'un programme.
- **rendre** le code plus lisible et plus clair en le fractionnant en blocs logiques.
- **pouvoir** appeler une fonction et donc exécuter les instructions qu'elle contient autant de fois qu'on le souhaite, ce qui constitue au final un gain de temps conséquent pour le développement d'un programme et ce qui nous permet de créer un code beaucoup plus clair.

A retenir

Chaque **fonction** effectue en général une tâche unique et précise. Si cela se complique, il est plus judicieux d'écrire plusieurs fonctions (qui peuvent éventuellement s'appeler les unes les autres). Cette modularité améliore la qualité générale et la lisibilité du code. Vous verrez qu'en Python, les fonctions présentent une grande flexibilité.



1.2. Type de fonctions

Il existe deux grands "types" de fonctions en Python :

- les **fonctions prédéfinies**,
- les **fonctions créées** par l'utilisateur.

1.3. Fonction prédéfinies (ou fonctions intégrées)

Python possède une série de **fonctions prédéfinies (ou fonctions intégrées)** afin de réaliser les fonctions les plus courantes.

Par exemple :

- `print()`
- `input()`
- `min()`, `max()`
- `sorted()`
- `len()`
- `range()`
- ...

Toutes ces fonctions peuvent être appelées directement depuis Python. Elles sont chargées au démarrage de Python.

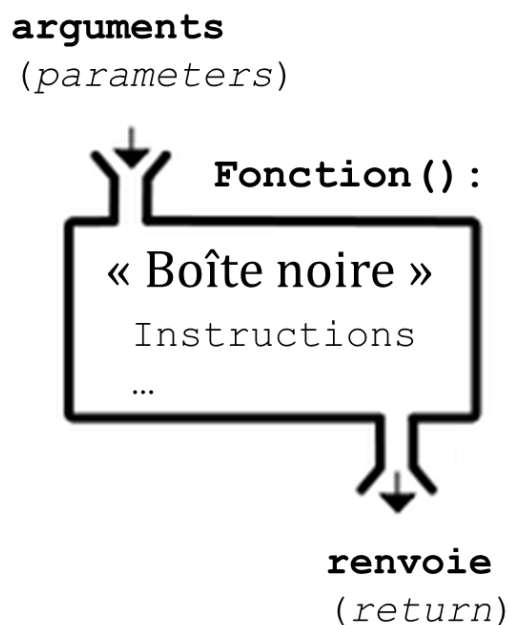
1.4. Fonction créées par l'utilisateur

Bien sûr, l'un des avantages de la programmation est de pouvoir créer sa propre fonction. C'est l'essence même de la programmation !

Par sa simplicité pour créer et utiliser les fonctions, Python est devenu l'un des langages les plus populaire mondialement.

1.5. Premiers principes des fonctions

Pour l'instant, une **fonction** est à nos yeux une sorte de « **boîte noire** », comme le montre la figure ci-dessous.



Représentation d'une fonction sous forme de « boîte noire »

Les principaux éléments de cette « boîte noire » sont :

1. on passe à la **fonction** une ou plusieurs variable(s) entre parenthèses (aucune variable est aussi acceptée). Ces variables sont appelées **arguments** (*parameters en anglais*). Il peut s'agir de n'importe quel type « d'objet » Python
2. on effectue un **ensemble d'instruction**
3. Et qui **renvoie** (*return en anglais*) un « objet » Python, ou rien du tout.

A retenir

Une **fonction** effectue une tâche. Pour cela, elle reçoit éventuellement des **arguments** et **renvoie** éventuellement quelque chose. L'algorithme utilisé au sein de la fonction n'intéresse pas directement l'utilisateur.

2. Fonctions prédéfinies

Les **fonctions prédéfinies** (*built-in-functions* en anglais) de Python sont des fonctions déjà disponibles par défaut, sans qu'il soit nécessaire d'importer un module. Elles permettent d'effectuer rapidement des opérations courantes.

2.1. Définition

Les fonctions prédéfinies sont des **outils standards** mis à disposition par Python pour :

- Manipuler les types de données (nombres, chaînes, listes, etc.),
- Effectuer des conversions,
- Réaliser des calculs simples,
- Interagir avec l'utilisateur ou le système.

Elles font partie du « **namespace builtins** » et sont donc utilisables partout.

2.2. Exemples de fonctions prédéfinies utiles

Fonction	Rôle	Exemple
<code>print()</code>	Affiche un message à l'écran	<code>print("Bonjour !")</code>
<code>input()</code>	Lit une entrée utilisateur (en chaîne)	<code>nom = input("Ton nom ? ")</code>
<code>len()</code>	Renvoie la longueur d'un objet (liste, chaîne, etc.)	<code>len("Python") # 6</code>
<code>type()</code>	Donne le type d'un objet	<code>type(42) # <class 'int'></code>
<code>int()</code> , <code>float()</code> , <code>str()</code>	Convertissent les types	<code>int("5") # 5</code>
<code>abs()</code>	Valeur absolue d'un nombre	<code>abs(-7) # 7</code>
<code>round()</code>	Arrondit un nombre	<code>round(3.14159, 2) # 3.14</code>
<code>max()</code> , <code>min()</code>	Renvoient le maximum ou minimum d'une séquence	<code>max([3, 5, 2]) # 5</code>
<code>sum()</code>	Somme des éléments d'un itérable	<code>sum([1, 2, 3]) # 6</code>
<code>sorted()</code>	Trie une séquence et retourne une nouvelle liste	<code>sorted([3, 1, 2]) # [1, 2, 3]</code>
<code>range()</code>	Génère une séquence de nombres	<code>list(range(3)) # [0, 1, 2]</code>

Où trouve t'on les fonctions prédéfinies ?

En Python, les **fonctions prédéfinies** (appelées aussi *built-in functions*) sont disponibles **par défaut**, sans avoir besoin d'importer de **module**.

Elles se trouvent dans le **module builtins**, qui est automatiquement chargé au démarrage de Python.

2.3. Qu'est-ce qu'un module ?

Un **module** (*module* en anglais) est simplement un fichier Python qui contient du code (fonctions, variables, classes).

Par exemple, dans le script Python ci-dessous, on utilise le module **math** pour afficher le résultat de la racine carrée de 16 (soit 4).

```
import math
print(math.sqrt(16)) # Résultat : 4.0
```

Le module **math** regroupe plusieurs fonctions mathématiques (sqrt (racine carrée), sin, cos, etc.) et doit être importé avec l'instruction **import** avant l'utilisation de la fonction choisie.

A savoir

Dans python, les modules peuvent être regroupés dans une bibliothèque

2.4. Qu'est-ce qu'une bibliothèque ?

Une **bibliothèque** (ou **package**) (*library* en anglais) est un **ensemble de modules** organisés ensemble pour accomplir une tâche plus large.

Par exemple, on utilise la bibliothèque **NumPy** pour le calcul scientifique, qui contient de nombreux modules.

```
# Importation de la bibliothèque NumPy
import numpy as np

# 1. Création d'un tableau NumPy
a = np.array([1, 2, 3, 4, 5])
print("Tableau a :", a)

# 2. Création d'une matrice 2x3 remplie de zéros
b = np.zeros((2, 3))
print("\nMatrice b (zéros) :\n", b)
```

La bibliothèque **numpy** est importé avec l'instruction **import**. On note que pour simplifier l'écriture, on codera la bibliothèque **numpy** -> **np** (en utilisant l'instruction **as np**)

A savoir

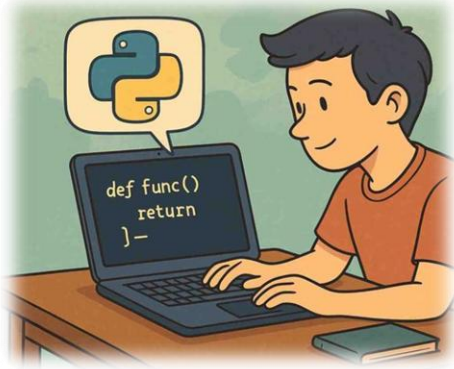
Dans Python, une **bibliothèque** est un ensemble logiciel de **modules** (*classes* (types d'objets), *fonctions*, *constantes*, ...) ajoutant des possibilités étendues à Python : calcul numérique, graphisme, programmation internet ou réseau, formatage de texte, génération de documents, ...

Il en existe un très grand nombre, et c'est d'ailleurs une des grandes forces de Python.

Les 9 meilleures bibliothèques Python

3. Création d'une fonction

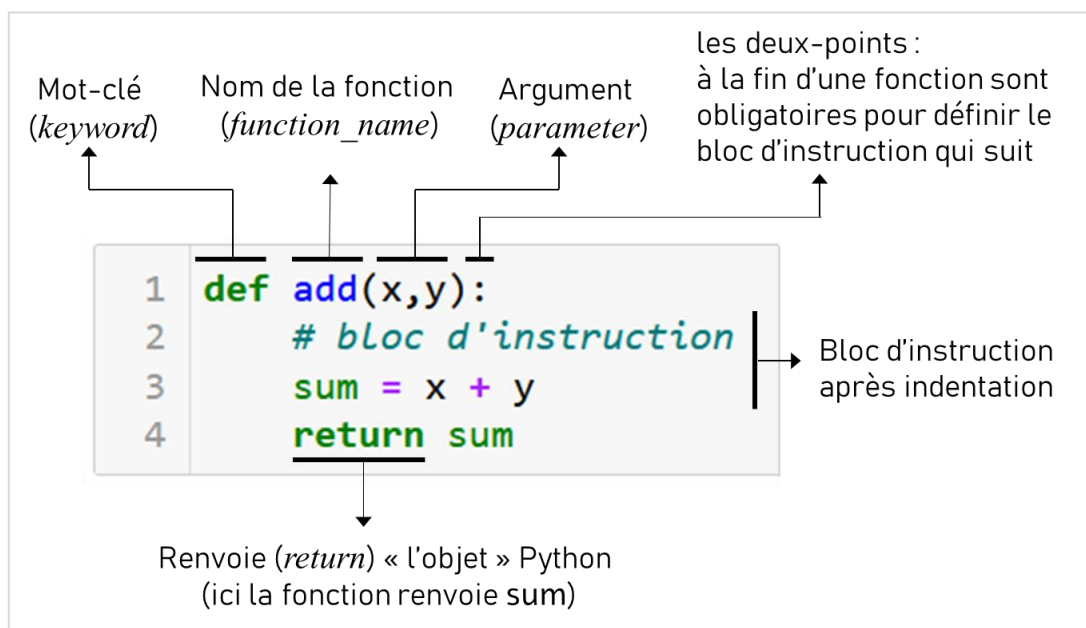
3.1. Syntaxe



Pour définir une fonction, Python utilise le mot-clé **def**. Puis, l'on utilise le **nom de la fonction** en passant les **arguments** (ou pas). Si l'on souhaite que la fonction **renvoie** quelque chose (pas obligatoire), il faut utiliser le mot-clé **return**.

Noter que la syntaxe de **def** utilise les **deux points** ;, ce qui signifie qu'un **bloc d'instructions** est attendu. L'**indentation** de ce bloc d'instructions (qu'on appelle le corps de la fonction) est obligatoire.

Un exemple de la syntaxe d'une fonction est donné ci-dessous :



Syntaxe d'une fonction en Python

En résumé, voici les éléments de syntaxe d'une fonction :

- **def** : définir la fonction
- **add** : nom de la fonction
- **()** : parenthèses pour **appeler** la fonction

- **x, y** : arguments placés entre les parenthèses de la fonction
- **:** (**deux points**) : indication de fin de déclaration de la fonction et attente d'un retour à la ligne et du début d'un bloc d'instruction après **indentation**.
- **sum = x + y** : Bloc d'instruction
- **return** : renvoie de la variable **sum**

2.2. Appel de la fonction

Dans l'exemple précédent, nous avons passé deux **arguments** à la fonction **add(x,y)** pour **renvoyer** (ou retourner) la valeur du calcul **x+y** via la variable **sum**.

Maintenant que la fonction est définie, on peut l'appeler autant de fois qu'on veut pour l'exécuter et récupérer le renvoie de la fonction (e.g. dans une variable ou pour afficher le résultat de renvoie).

Ici, on définit deux variables **a = 2** et **b = 3**, puis on appelle la fonction **add(a,b)** dont le renvoie est affecté dans la variable **result**, laquelle peut être affichée à l'écran à l'aide de la fonction **print()** :

```

1  def add(x,y):
2      # bloc d'instruction
3      sum = x + y
4      return sum
5
6  a = 2
7  b = 3
8  result = add(a,b)
9
10 print(f"le résultat de {a} + {b} est : {result}")

```

le résultat de 2 + 3 est : 5

Noter qu'une fonction ne prend pas forcément un argument et ne renvoie pas forcément une valeur, par exemple :

```

1  def hello():
2      print("bonjour")
3
4  hello()

```

bonjour

Dans ce cas la fonction, **hello()** se contente d'afficher la *chaîne de caractères* "**bonjour**" à l'écran. Elle ne prend aucun argument et ne renvoie rien. Par conséquent, cela n'a pas de sens de vouloir récupérer dans une variable le résultat renvoyé par une telle fonction.

Si on essaie tout de même, Python affecte la valeur **None** qui signifie *rien* en anglais :


```
1 var = hello()
bonjour
```

```
1 print(var)
None
```

Ceci n'est pas une faute car Python n'émet pas d'erreur, toutefois cela ne présente, la plupart du temps, guère d'intérêt.

3. Passage d'arguments dans une fonction

3.1. Multiples arguments

Le nombre d'**arguments** que l'on peut passer à une **fonction** est variable. Nous avons vu ci-dessus des fonctions auxquelles on passait 0 ou 1 argument. Cependant, le nombre d'argument est laissé libre à l'initiative du programmeur qui développe une nouvelle fonction. On peut donc faire passer 2, 3 ou plus d'arguments à une fonction.

Une particularité des fonctions en Python est que vous n'êtes pas obligé de préciser le **type** des arguments que vous lui passez, dès lors que les opérations que vous effectuez avec ces arguments sont valides. Python est en effet connu comme étant un langage au « **typage dynamique** », c'est-à-dire qu'il reconnaît pour vous le type des variables au moment de l'exécution.

3.2. Exemples

Utilisation de la fonction **fois()** qui retourne le produit de **2 arguments** passés à la fonction :

```
1 def fois(x, y):
2     return x*y
3
4 fois(2, 3)
6
```

```
1 fois(3.1415, 5.23)
16.430045000000003
```

```
1 fois('to', 2)
'toto'
```

On note que l'opérateur « * » reconnaît plusieurs types (*entiers, floats, chaînes de caractères*, etc.). Notre fonction **fois()** est donc capable d'effectuer des tâches différentes !

Même si Python autorise cela, méfiez-vous tout de même de cette grande flexibilité qui pourrait conduire à des surprises dans vos futurs programmes.

En général, il est plus judicieux que chaque argument ait un type précis (*entiers, floats, chaînes de caractères*, etc.) et pas l'un ou l'autre.

4. Renvoi de résultat dans une fonction

4.1. Renvoi multiple

Un énorme avantage en Python est que les fonctions sont capables de **renvoyer plusieurs objets à la fois**, comme dans cette fraction de code :

```
1 def carre_cube2(x):  
2     return x**2, x**3  
3  
4 carre_cube2(3)
```

(9, 27)

Remarque

Pour le renvoi multiple utiliser dans l'exemple ci-dessus, le résultat d'exécution est un type de donnée nommé **tuple** en Python (9, 27), lequel contient les deux valeurs (carré, cube).

Nous n'étudierons pas les **tuples** dans ce cours mais comme on peut le voir, c'est une représentation (a, b) généralement appelée **couple ordonné** ou **paire ordonnée** en mathématiques.

4.2. Affectation multiple

En Python, l'**affectation multiple** (ou *multiple assignent*) est une façon concise d'assigner plusieurs variables en une seule instruction.

Principe :

On sépare les variables par des virgules à gauche de = et on sépare les valeurs correspondantes à droite.

Par exemple :

```
1 z1, z2 = carre_cube2(3)
```

```
1 z1
```

9

```
1 z2
```

27

Cela permet de récupérer plusieurs valeurs renvoyées par une fonction et de les affecter à la volée à des variables différentes

5. Arguments positionnels et arguments par mot-clé dans une fonction

5.1. Arguments positionnels

Jusqu'à maintenant, nous avons systématiquement passé le nombre d'arguments que la fonction attendait. Par exemple :

```
1 def fois(x, y):
2     return x*y
3
4 fois(2, 3)
```

6

Que se passe-t-il si une fonction attend **deux arguments** et que nous ne lui en passons qu'un seul ?

```
1 fois(2)
```

```
--
--
TypeError                                Traceback (most recent call las
t)
<ipython-input-3-c4a1636488a1> in <module>
----> 1 fois(2)

TypeError: fois() missing 1 required positional argument: 'y'
```

On constate que passer un seul argument à une fonction qui en attend deux conduit à une **erreur**.

Définition

Lorsqu'on définit une fonction **fct(x, y)**: les arguments x et y sont appelés arguments positionnels. Il est strictement obligatoire de les préciser lors de l'appel de la fonction. De plus, il faut respecter le même ordre lors de l'appel que dans la définition de la fonction. Dans l'exemple, 2 correspondra à x et 3 correspondra à y. Finalement, tout dépendra de leur position, d'où leur qualification de positionnel.

5.2. Arguments par mot-clé

Il est aussi possible de passer un ou plusieurs argument(s) de manière facultative et de leur attribuer une valeur par défaut avec la syntaxe :

def fct(arg=val): où on attribue une valeur d'argument **arg=val** à la fonction **fct()**.

Définition

Un argument défini avec une syntaxe **def fct(arg=val):** est appelé argument par mot-clé. Le passage d'un tel argument lors de l'appel de la fonction est facultatif. Ce type d'argument ne doit pas être confondu avec les arguments positionnels présentés ci-dessus, dont la syntaxe est **def fct(arg):**

Par exemple :

```
1 def fct(x=1):  
2     return x  
3  
4 fct()
```

1

```
1 fct(10)
```

10

Il est bien sûr possible de passer plusieurs arguments par mot-clé :

```
1 def fct(x=0, y=0, z=0):  
2     return x, y, z  
3  
4 fct()
```

(0, 0, 0)

```
1 fct(10)
```

(10, 0, 0)

```
1 fct(10, 8)
```

(10, 8, 0)

```
1 fct(10, 8, 3)
```

(10, 8, 3)

On observe que pour l'instant, les **arguments** par **mot-clé** sont pris dans l'ordre dans lesquels on les passe lors de l'appel.

Comment pourrions-nous faire si on souhaitait préciser l'argument par mot-clé `z` et garder les valeurs de `x` et `y` par défaut ? Simplement en précisant le nom de l'argument lors de l'appel :

```
1 fct(z=10)
```

(0, 0, 10)

Python permet même de rentrer les **arguments** par **mot-clé** dans un ordre arbitraire :

```
1 fct(z=10, x=3, y=80)
```

(3, 80, 10)

```
1 fct(z=10, y=80)
(0, 80, 10)
```

Que se passe-t-il lorsque nous avons un mélange d'**arguments positionnels** et par **mot-clé** ? Et bien les arguments positionnels doivent toujours être placés avant les arguments par mot-clé :

```
1 def fct(a, b, x=0, y=0, z=0):
2     return a, b, x, y, z
3
4 fct(1, 1)
(1, 1, 0, 0, 0)
```

```
1 fct(1, 1, z=5)
(1, 1, 0, 0, 5)
```

```
1 fct(1, 1, z=5, y=32)
(1, 1, 0, 32, 5)
```

On peut toujours passer les arguments par mot-clé dans un ordre arbitraire à partir du moment où on précise leur nom. Par contre, si les deux arguments positionnels a et b ne sont pas passés à la fonction, Python renvoie une erreur :

```
1 fct(z=0)
```

```
TypeError                                Traceback (most recent call last)
<ipython-input-19-a12949074649> in <module>
----> 1 fct(z=0)

TypeError: fct() missing 2 required positional arguments: 'a' and 'b'
```

Conseil pratique

Préciser le nom des arguments par mot-clé lors de l'appel d'une fonction est une pratique que nous vous recommandons. Cela les distingue clairement des arguments positionnels.

L'utilisation d'arguments par mot-clé est habituelle en Python. Elle permet de modifier le comportement par défaut de nombreuses fonctions.

Par exemple, si on souhaite que la fonction `print()` n'affiche pas un retour à la ligne, on peut utiliser l'argument `end` :

```
1 print("Message ", end="")
Message
```

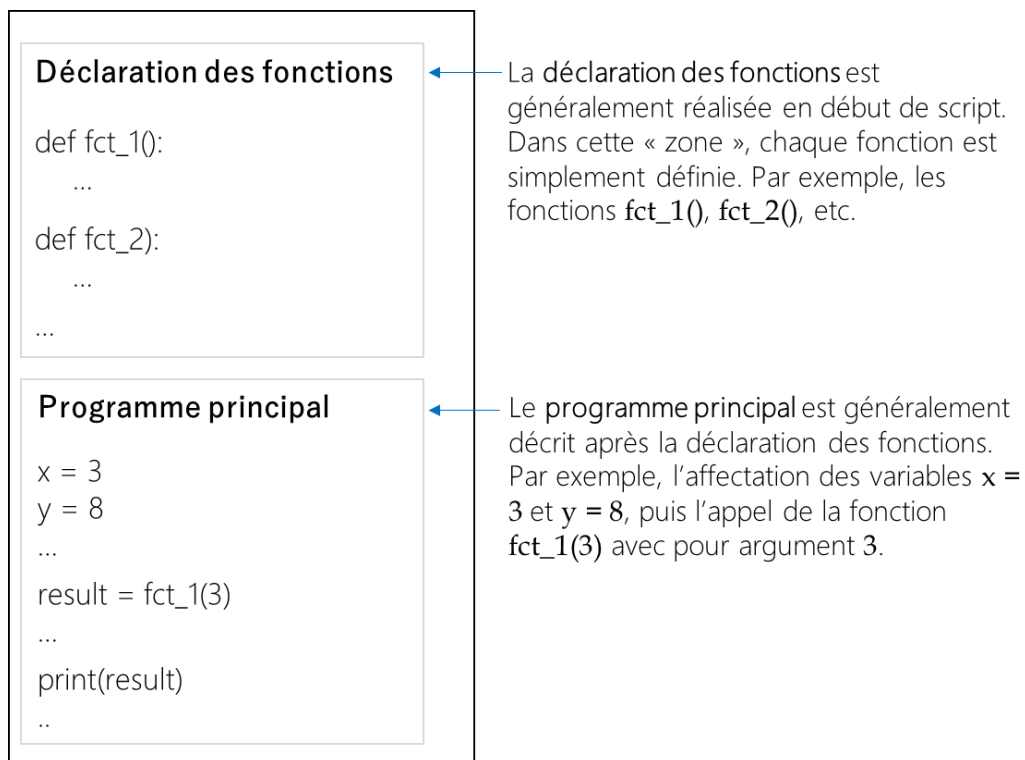
6. Déclaration des fonctions dans une application Python

6.1. Déclaration des fonctions et programme principal

En général, dans un script Python, on écrit d'abord les **fonctions** puis le **programme principal**. On parlera de « zone » de **déclaration des fonctions**.

On désignera comme **programme principal** (*main program* en anglais) l'endroit depuis lequel on appelle une **fonction** (on verra plus tard que l'on peut en fait appeler une fonction de n'importe où). Le programme principal désigne le code qui est exécuté lorsqu'on lance le script Python, c'est-à-dire toute la suite d'instructions en dehors des fonctions.

La figure ci-dessous présente l'organisation de votre programme Python intégrant les parties de déclaration des fonctions et du programme principal.



Exemple d'une architecture de script Python, intégrant la « zone » de déclaration des fonctions et le programme principal.

A noter qu'il n'est pas obligatoire de commencer votre script par une « zone » de déclaration des fonctions. En effet, le programme principal peut également être placé avant puis après la « zone » de déclaration des fonctions.

Par contre, **il est nécessaire qu'une fonction soit définie avant d'être appelée** ! C'est pour cela qu'il est souvent commode de débiter par la déclaration de toutes les fonctions.

Dans le cas de la fonction `carre_cube2(x)` présentée précédemment, on voit bien que si la fonction n'est pas définie avant d'être appelée, l'interpréteur Python renvoie :

NameError : name 'carre_cube2' is not defined.

```

1 carre_cube2(3)
2
3 def carre_cube2(x):
4     return [x**2, x**3]

```

```

-----
NameError                                Traceback (most recent call last)
<ipython-input-1-2eab61bae76a> in <module>
----> 1 carre_cube2(3)
      2
      3 def carre_cube2(x):
      4     return [x**2, x**3]

NameError: name 'carre_cube2' is not defined

```

6.2. Exemple

Un exemple de de script Python est donné ci-dessous, avec les parties de déclaration des fonctions et du programme principal.

On remarque que même sans connaître les fonctions du script, on peut identifier sans mal la zone de déclaration des fonctions et celle du programme principal d'où sont appelées les fonctions.

Zone de déclaration des fonctions qui seront appelées dans le programme principal

```

#function for dice roll
def roll_dice():
    num1 = random.randint(1,6)
    num2 = random.randint(1,6)
    return num1, num2

#function for outputting the numbers rolled to the user
def user_roll(num1, num2):
    print("You have rolled", num1, "and", num2)

```

Programme principal d'où sont appelées les fonctions définies en début de script

```

roll_dice()
user_roll(roll_dice())

```

Exemple d'un script Python où l'on reconnaît la « zone » de déclaration des fonctions et la partie programme principal d'où sont appelées les fonctions.

7. Variables globales et variables locales d'une fonction

7.1. Définition

Lorsqu'on manipule des **fonctions**, il est essentiel de bien comprendre comment se comportent les **variables** et de connaître la différence entre variable dite **globale** et variable dite **locale**.

Définitions

- **Variable globale :**
Une variable est dite **globale** lorsqu'elle est créée dans le programme principal (*main program*). Elle sera visible partout dans le programme.
- **Variable locale :**
Une variable est dite **locale** lorsqu'elle est créée dans une fonction (*function*). Elle n'existera et ne sera visible que lors de l'exécution de ladite fonction.

Exemple de script Python donnée ci-dessous

```
1  # Script Python
2
3  # Déclaration des fonctions
4
5  def fct() :
6      x = 10
7      y = 20
8
9  # Programme principal
10
11 x = 1
12 y = 2
13
14 fct()
15
16 print(f"La variable x est égale à {x}")
17 print(f"La variable y est égale à {y}")
```

Variables locales
(x, y variables propres à la fonction fct())

Variables globales
(x, y propres au programme principal)

```
La variable x est égale à 1
La variable y est égale à 2
```

Dans la déclaration des fonctions, on définit une fonction `fct()`, laquelle instruit deux **variables dites locales** `x` et `y`. Dans le programme principal, on déclare deux **variables dites globales** `x` et `y`.

Attention

Même si les variables `x` et `y` sont nommées de manière identique dans la fonction `fct()` et le programme principal, elles ne sont pas identiques !

En effet, en exécutant le script Python, on voit que les variables `x = 10` et `y=20` restent des **variables locales** utilisées dans la fonction `fct()`. Elles ne sont jamais renvoyées dans le programme principal, même si la fonction `fct()` est appelée.

Finalement, les **variables globales** déclarées dans le programme principal `x = 1` et `y=2` sont celles que l'on affichera après exécution du script.

7.2. Exemple

Ceci ne vous paraît pas clair ? Nous allons prendre et détailler un exemple simple qui vous aidera à mieux saisir ces concepts. Observez le code suivant :

```

1  # définition d'une fonction carre()
2  def carre(x):
3      y = x**2
4      return y
5
6  # programme principal
7  z = 5
8  result = carre(z)
9  print(result)

```

Nous allons utiliser l'outil **Python Tutor** pour suivre les 9 étapes d'exécution du script précédent.

Python Tutor est un outil permet de visualiser en live le déroulement d'un programme Python. De plus, il présente l'ensemble des variables dans un graphique très intuitif. C'est un outil pédagogique de choix qui de plus ne requiert aucune installation !

Lien du site : <http://www.pythontutor.com>

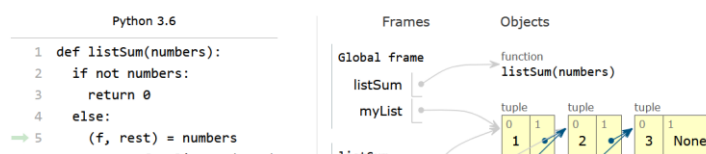
Online Compiler, AI Tutor, and Visual Debugger for Python, Java, C, C++, and JavaScript

Python Tutor helps you do **programming homework assignments** in Python, Java, C, C++, and JavaScript. It contains a step-by-step [visual debugger](#) and [AI tutor](#) to help you understand and debug code.

Start coding online in [Python](#), [Java](#), [C](#), [C++](#), and [JavaScript](#)

Since 2010, **over 20 million people in more than 180 countries** have used Python Tutor to visualize over 300 million pieces of code. It is the most widely-used program visualization tool for CS education.

As a preview, here is a [small example](#) that visualizes recursion in Python:



Détail de chaque étapes d'exécution du script via Python Tutor

ETAPE 1 :

- Python est prêt à lire la première ligne de code

Python 3.6

```

1 # définition d'une fonction carre()
2 def carre(x):
3     y = x**2
4     return y
5
6 # programme principal
7 z = 5
8 resultat = carre(z)
9 print(resultat)

```

Print output (drag lower right corner to resize)

Frames Objects

ETAPE 2 :

- Python met en mémoire la fonction **carre()**. Notez qu'il ne l'exécute pas ! La fonction est mise dans un espace de la mémoire nommé *Global frame*, il s'agit de l'espace du programme principal. Dans cet espace, seront stockées toutes les **variables globales** créées dans le programme. Python est maintenant prêt à exécuter le programme principal

Python 3.6

```

1 # définition d'une fonction carre()
2 def carre(x):
3     y = x**2
4     return y
5
6 # programme principal
7 z = 5
8 resultat = carre(z)
9 print(resultat)

```

Print output (drag lower right corner to resize)

Frames Objects

Global frame

carre

function
carre(x)

ETAPE 3 :

- Python lit et met en mémoire la variable **z**. Celle-ci étant créée dans le programme principal, il s'agira d'une **variable globale**. Ainsi, elle sera également stockée dans le *Global frame*.

Python 3.6

```

1 # définition d'une fonction carre()
2 def carre(x):
3     y = x**2
4     return y
5
6 # programme principal
7 z = 5
8 resultat = carre(z)
9 print(resultat)

```

Print output (drag lower right corner to resize)

Frames Objects

Global frame

carre

z 5

function
carre(x)

ETAPE 4 :

- La fonction **carre()** est appelée et on lui passe en argument l'entier **z**. La fonction s'exécute et un nouveau cadre est créé dans lequel Python Tutor va indiquer toutes les **variables locales** à la fonction. Notez bien que la variable passée en argument, qui s'appelle **x** dans la fonction, est créée en tant que **variable locale**. On remarquera aussi que les **variables globales** situées dans le *Global frame* sont toujours là.

Python 3.6

```

1 # définition d'une fonction carre()
2 def carre(x):
3     y = x**2
4     return y
5
6 # programme principal
7 z = 5
8 resultat = carre(z)
9 print(resultat)

```

Print output (drag lower right corner to resize)

Frames

Global frame

carre	
z	5

Objects

function carre(x)

carre

x	5
---	---

ETAPE 5 :

- Python est maintenant prêt à exécuter chaque ligne de code de la fonction.

Python 3.6

```

1 # définition d'une fonction carre()
2 def carre(x):
3     y = x**2
4     return y
5
6 # programme principal
7 z = 5
8 resultat = carre(z)
9 print(resultat)

```

Print output (drag lower right corner to resize)

Frames

Global frame

carre	
z	5

Objects

function carre(x)

carre

x	5
---	---

ETAPE 6 :

- La variable **y** est créée dans la fonction. Celle-ci est donc stockée en tant que **variable locale** à la fonction.

Python 3.6

```

1 # définition d'une fonction carre()
2 def carre(x):
3     y = x**2
4     return y
5
6 # programme principal
7 z = 5
8 resultat = carre(z)
9 print(resultat)

```

Print output (drag lower right corner to resize)

Frames

Global frame

carre	
z	5

Objects

function carre(x)

carre

x	5
y	25

ETAPE 7 :

- Python s'apprête à renvoyer la **variable locale y** au programme principal. Python Tutor nous indique le contenu de la valeur renvoyée.

→ line that just executed
→ next line to execute

Python 3.6

```

1 # définition d'une fonction carre()
2 def carre(x):
3     y = x**2
4     return y
5
6 # programme principal
7 z = 5
8 resultat = carre(z)
9 print(resultat)

```

Print output (drag lower right corner to resize)

Frames

Global frame	
carre	
z	5

Objects

carre	
x	5
y	25
Return value	25

ETAPE 8 :

- Python quitte la fonction et la **valeur renvoyée** par celle-ci est affectée à la **variable globale resultat**. Notez bien que lorsque Python quitte la fonction, l'espace des variables alloué à la fonction est détruit. Ainsi, toutes les variables créées dans la fonction n'existent plus. On comprend pourquoi elles portent le nom de locales puisqu'elles n'existent que lorsque la fonction est exécutée.

→ line that just executed
→ next line to execute

Python 3.6

```

1 # définition d'une fonction carre()
2 def carre(x):
3     y = x**2
4     return y
5
6 # programme principal
7 z = 5
8 resultat = carre(z)
9 print(resultat)

```

Print output (drag lower right corner to resize)

Frames

Global frame	
carre	
z	5
resultat	25

Objects

function carre(x)	
-------------------	--

ETAPE 9 :

- Python affiche le contenu de la variable **resultat** et l'exécution est terminée.

→ line that just executed
→ next line to execute

Python 3.6

```

1 # définition d'une fonction carre()
2 def carre(x):
3     y = x**2
4     return y
5
6 # programme principal
7 z = 5
8 resultat = carre(z)
9 print(resultat)

```

Print output (drag lower right corner to resize)

25

Frames

Global frame	
carre	
z	5
resultat	25

Objects

function carre(x)	
-------------------	--

8. Exercices

8.1. Fonction sans argument : `bonjour()`

Écrire une fonction **`bonjour()`** qui affiche simplement « Bonjour ! ». Appeler ensuite cette fonction.

8.2. Fonction avec argument : `saluer(nom)`

Écrire une fonction **`saluer(nom)`** qui prend un argument `nom` et affiche « Bonjour, nom ! ».

8.3. Fonction avec retour de valeur

Écrire une fonction **`carre(x)`** qui retourne le carré du nombre `x`. Afficher ensuite le résultat pour `x = 7`.

8.4. Fonction avec plusieurs paramètres

Écrire une fonction **`additionner(a, b)`** qui retourne la somme de `a` et `b`. Tester-la avec `a = 12` et `b = 8`.

8.5. Paramètre par défaut

Créer une fonction **`salutation(nom, message="Bonjour")`** qui affiche `message`, `nom` !

Testez-la avec :

- seulement le nom
- le nom et un message personnalisé

8.6. Fonction qui renvoie plusieurs valeurs

Écrire une fonction **`operations(a, b)`** qui retourne **la somme et le produit** de `a` et `b`. Afficher ensuite les deux résultats séparément.

Corrections

8.1. Fonction sans argument : bonjour()

```
def bonjour():  
    print("Bonjour !")  
  
# Appel de la fonction  
bonjour()
```

Explication : Ici, la fonction n'a pas de paramètres et exécute toujours la même instruction.

8.2. Fonction avec argument : saluer(nom)

```
def saluer(nom):  
    print(f"Bonjour, {nom} !")  
  
# Appel de la fonction  
saluer("Alice")  
saluer("Bob")
```

Explication : Ici, l'argument permet de rendre la fonction réutilisable avec différentes valeurs.

8.3. Fonction avec retour de valeur

```
def carre(x):  
    return x ** 2  
  
# Utilisation de la fonction  
resultat = carre(7)  
print("Le carré de 7 est :", resultat)
```

Explication : Ici, le **return** permet de récupérer une valeur et de l'utiliser ailleurs dans le code..

8.4. Fonction avec plusieurs paramètres

```
def additionner(a, b):  
    return a + b  
  
print("12 + 8 =", additionner(12, 8))
```

Explication : Ici, on passe plusieurs arguments à la fonction et on récupère le résultat via **return**.

8.5. Paramètre par défaut

```
def salutation(nom, message="Bonjour"):  
    print(f"{message}, {nom} !")  
  
salutation("Sophie")  
salutation("Lucas", "Bienvenue")
```

Explication : Ici, les paramètres par défaut permettent d'appeler la fonction sans fournir toutes les valeurs.

8.6. Fonction qui renvoie plusieurs valeurs

```
def operations(a, b):  
    somme = a + b  
    produit = a * b  
    return somme, produit  
  
s, p = operations(3, 4)  
print("Somme :", s)  
print("Produit :", p)
```

Explication : Ici, une fonction peut renvoyer plusieurs valeurs sous forme de **tuple** (le type de donnée tuple n'est pas étudiée dans ce cours mais présenté à titre d'exemple mathématique)